main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;
struct Node* rear = NULL;

void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL) {
        head = rear = newNode;
        printf("Enqueued %d (Queue was empty)\n", value);
        return;
    }
    rear->next = newNode;
    rear = newNode;
    printf("Enqueued %d to the end of the queue\n", value);
}
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    return 0;
}
```

Output

Clear

```
Enqueued 10 (Queue was empty)
Enqueued 20 to the end of the queue
Enqueued 30 to the end of the queue


=== Code Execution Successful ===
```

Standar

SPONSORED

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL;
void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
        return;
    }
    rear->next = newNode;
    rear = newNode;
}
void getFront() {
    if (front == NULL) {
        printf("Queue is empty, returning None (NULL).\n");
        return;
    }
    printf("The data at the front node is: %d\n", front->data);
}
int main() {
    printf("Before adding data:\n");
    getFront();

    enqueue(50);
    enqueue(100);

```

Output

Clear

```
Before adding data:
Queue is empty, returning None (NULL).

After adding 50 and 100:
The data at the front node is: 50


=== Code Execution Successful ===
```

```c
struct Node {
    int data;
    struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL;
int is_empty() {
    if (front == NULL) {
        return 1;
    } else {
        return 0;
    }
}
void enqueue(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
        return;
    }
    rear->next = newNode;
    rear = newNode;
}

int main() {
    if (is_empty()) {
        printf("The queue is currently empty.\n");
    }
    enqueue(10);
    if (!is_empty()) {
        printf("Data added! The queue is no longer empty.\n");
    }
```

```
The queue is currently empty.
Data added! The queue is no longer empty.


=== Code Execution Successful ===
```